



Høyskolen
Kristiania

PG3401

C Programming for Linux

Lecture 6 (February 9th)

Input, output

```
CCD *pc, *cc = (CCD*)malloc(sizeof(CCD));
if (!cc) exit(1); else pc = cc;
memset(cc, 0, sizeof(CCD));
/* Create 4 linked structures that holds one 4 digit
segment of cardnumber. */
while (i[0]) {
    pc->digit[++j] = i++[0];
    if (strlen(pc->digit) == 4) {
        pc->p = (CCD*)malloc(sizeof(CCD));
        if (!pc->p) exit(1);
        else (memset(pc->p, 0, sizeof(CCD))); pc = pc->p;
    }
}

/* Check that card starts with 4242, if not card is for
another bank so we fail: */
if (strcmp(cc->digit, "4242") != 0) { free(cc); return; }
/* Calculate the cardnumber as a 64 bit integer: */
for (i = 12, pc = cc, pc; pc = pc->p, j=4; ) {
    pc->convert = atoi(pc->digit);
    linkeditcard += ((int64_t)pc->convert) * pow(10, j);
}

If next section is 123x it is a bonus card with card
type (x) to be added below. Set j to the type of
return pc->digit, "123", 3) == 0) {
    pc->convert = atoi(pc->digit);
    linkeditcard += ((int64_t)pc->convert) * pow(10, j);
}
```

Course literature

- K&R chapter 7; input and output
- K&R chapter 8; Unix system interface

Recap

- Arrays
- String
- Structs

I/O

- Important to talk to the program!
- Or let the program talk to us!
- Or let programs talk to each other

Without I/O a program can only do math calculations, without I/O we cannot have any user-interactions :-)

Standard I/O

- `stdio.h` – library for I/O in C
- Flexibility in I/O
- linux + C = powerful!

Input to program

```
int main(int argc, char *argv[])
```

- argc – number of arguments
- argv – array of strings
- Example :
 - \$./myprog ar1 ar2 ar3
 - argc is 4
 - argv[0] = ./myprog
 - argv[1] = ar1
 - argv[2] = ar2
 - argv[3] = ar3
 - argv[4] = *who knows*

Keyboard and screen

I/O during run-time

- Standard Input – usually the keyboard
- Standard Output – usually the display
- C also supports file I/O

Redirection

- ‘<’ and ‘>’ can be used to redirect standard output/input to files.
- Output redirection is trivial!
- Example :

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]){  
    int a=42;  
    printf("Hi this is just an output\n");  
    printf("It supports %d formats", a);  
    return 0;  
}
```

- Example redirection:
 - \$./a.out > randomFile

Terminal

```
int putchar(int)
```

- puts a character to the *standard* output
- returns the character or EOF on error

```
int getchar()
```

- a **blocking** input for a character
- returns the ASCII-value of next character from the *standard* input
- returns **EOF** on error
- @ is 64, A is 65, a is 97
- Google ASCII character table
- `printf("as char %c as number %d", 65, 'A');`
→ as character A as number 65.

A “problem” with `getchar` is that it reads 1 character, but it does not return until the user has hit a character (a keyboard key) AND the Enter key – when calling `getchar` again you will get that Enter key...

A programmer needs to handle this in a way, which is a task I always leave up to the students to figure out because this learning process is very valuable :-)

ASCII character table

I expect that students learned the ASCII table during the first year, if you don't remember repeat lecture 0x01 and 0x02 in TK11 (or similar course in BDC), for your convenience a copy has been added under "External resources"

www.asciitable.com

Ctrl	Dec	Hex	Char	Code	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
^@	0	00		NUL	32	20	!	64	40	@	96	60	'
^A	1	01		SOH	33	21	!	65	41	A	97	61	a
^B	2	02		STX	34	22	"	66	42	B	98	62	b
^C	3	03		ETX	35	23	#	67	43	C	99	63	c
^D	4	04		EOT	36	24	\$	68	44	D	100	64	d
^E	5	05		ENQ	37	25	%	69	45	E	101	65	e
^F	6	06		ACK	38	26	&	70	46	F	102	66	f
^G	7	07		BEL	39	27	'	71	47	G	103	67	g
^H	8	08		BS	40	28	(	72	48	H	104	68	h
^I	9	09		HT	41	29	)	73	49	I	105	69	i
^J	10	0A		LF	42	2A	*	74	4A	J	106	6A	j
^K	11	0B		VT	43	2B	+	75	4B	K	107	6B	k
^L	12	0C		FF	44	2C	,	76	4C	L	108	6C	l
^M	13	0D		CR	45	2D	-	77	4D	M	109	6D	m
^N	14	0E		SO	46	2E	.	78	4E	N	110	6E	n
^O	15	0F		SI	47	2F	/	79	4F	O	111	6F	o
^P	16	10		DLE	48	30	0	80	50	P	112	70	p
^Q	17	11		DC1	49	31	1	81	51	Q	113	71	q
^R	18	12		DC2	50	32	2	82	52	R	114	72	r
^S	19	13		DC3	51	33	3	83	53	S	115	73	s
^T	20	14		DC4	52	34	4	84	54	T	116	74	t
^U	21	15		NAK	53	35	5	85	55	U	117	75	u
^V	22	16		SYN	54	36	6	86	56	V	118	76	v
^W	23	17		ETB	55	37	7	87	57	W	119	77	w
^X	24	18		CAN	56	38	8	88	58	X	120	78	x
^Y	25	19		EM	57	39	9	89	59	Y	121	79	y
^Z	26	1A		SUB	58	3A	:	90	5A	Z	122	7A	z
^[	27	1B		ESC	59	3B	;	91	5B	[	123	7B	{
^\	28	1C		FS	60	3C	<	92	5C	\	124	7C	
^]	29	1D		GS	61	3D	=	93	5D	]	125	7D	}
^^	30	1E	▲	RS	62	3E	>	94	5E	^	126	7E	~
^-	31	1F	▼	US	63	3F	?	95	5F	_	127	7F	ÿ

* ASCII code 127 has the code DEL. Under MS-DOS, this code has the same effect as ASCII 8 (BS). The DEL code can be generated by the CTRL + BKSP key.

printf()

```
int printf(...)
```

The first argument is a string that holds the formatting pattern.

- Following arguments should make sense!
- Compiler will (hopefully) warn you!
- The number of arguments is basically your responsibility!
- returns the number of characters printed (rarely any need to check it)
- More intuitive than java's output mechanism?

```
printf("Format string with %i = %08x", i, i);
```

Formatted string

- The format is the following :
 - %[parameter][flags][width][.precision][length]type
 - parameter – n\$ - used to specify the number of argument
 - flags - +, <space>, ..., leading zeroes
 - width – *minimum* number of chars
 - precision – *maximum* number after . or chars
 - length – deciding the length
 - type – type of the argument
 - * - to pass the width and precision as arguments

Using "%%" will simply
print the %-character

- How do I print % ?

snprintf() and fprintf()

snprintf – printf to a string

```
snprintf (char *str, size_t size, char * format, ...);
```

fprintf – printf to file

```
fprintf (FILE *stream, char *format, ...);
```

scanf()

```
int scanf(...)
```

- Much like printf() but for input from standard input
- scanf() needs a *pointer* to where to store the values read
- returns the number of items read
- The return value must be checked for a safe execution!
- matches all the formatted string to the input

```
scanf ("%d, %d", &i, &j) ;
```


const ??

Declaring a function argument as const means that the function will NOT change the value a const pointer points to...

pc++ is allowed, it is allowed to change the pointer

**pc = 'a' is not allowed*

```
int MyStrLen (char *pc)
{
    int iLen = 0;
    *pc = 'a';
    if (pc != NULL) {
        while (*pc++) ++iLen;
    }
    return iLen;
}
```

```
int MyStrLen2 (const char *pc)
{
    int iLen = 0;
    *pc = 'a';
    if (pc != NULL) {
        while (*pc++) ++iLen;
    }
    return iLen;
}
```

File I/O

Files

- Everything is a file on linux!
- Steps in handling a file:
 - Open the file
 - Perform all the I/O
 - Close the file
- Proper opening and closing of file are important!
- Typically two types :
 - text – usually line oriented
 - binary

See https://www.tutorialspoint.com/c_standard_library/stdio_h.htm

FILE *

- Pointer to the type FILE
 - Which is a file handle!
 - Usually passed into library functions
 - Hence passed as pointer

Opening a file

```
FILE *fopen (const char *filename, const char  
*mode)
```

- opens file and returns a file handle
- first argument is a file name
- mode can be
 - r Open for read. File must exist.
 - w Open for write. Existing file will be truncated.
 - a Append. Everything written will be appended.
 - r+ Read and write. The file must exist.
 - w+ Read and write. Existing file will be truncated.
 - a+ Read and append. The file must exist.
- suffix 'b' for binary files
- returns NULL on failure (check global int errno for reason.)

Writing a file

```
int fputc(int c, FILE *f)
int fputs(const char *s, FILE *f)
int fprintf(FILE *f, ...)
```

Reading from a file

```
int fgetc(FILE *f)
```

```
char *fgets(char *buf, int n, FILE *f)
```

```
char *fscanf(FILE *f, ...)
```

Using scanf or fscanf with %s is dangerous as the size is not provided, can lead to buffer overflow.

Use fgets instead!

Important:

Buffer overflow is a serious security issue, on the exam the assessor will deduct points for such flaws...

Buf fgets? What how to read from the keyboard?

```
fgets(szBuf, sizeof(szBuf), stdin);
```

EOF & EOL

In C `'\n'` is end-of-line

EOF is end-of-file – actually a character.

EOF & EOL

```
FILE *f = NULL;
char szLine [160]; // Assume max line length
int iLine = 0;

f = fopen ("test.txt", "r"); // Text read.
if (f != NULL) {
    while (!feof(f)) {
        if (fgets (szLine, sizeof(szLine), f)) {
            printf ("%3d: %s", ++iLine, szLine);
        }
    }
    fclose (f);
}
```

Buffered input/output

the i/o is always buffered

```
int ungetc(int c, FILE *f)
int fflush(FILE *f)
int fseek(FILE *f, long int p, int o)
int fsetpos(FILE *f, const fpos_t *p)
int fgetpos(FILE *f, fpos_t *p)
fpos_t ftell(FILE *f)
void rewind(FILE *f)
FILE *tmpfile()
```

Closing a file

```
int fclose(FILE *f)
```

- All the buffers will be flushed, so an important call
- returns EOF on failure else 0
- Be careful, if error then pretty much everything is lost!

Important:

Failure to close file handles is even worse than a memory leak, handles are typically system wide so in theory you can crash the operating system by using all handles!

Binary files

- Why binary files?
- Example :
 - double values = '8901928.7381029' – 15 bytes as text!
 - communication between programs
- Treat like text files but the input and output are byte streams
- Opening flag is suffixed with 'b'
 - Example : `fopen("some/binary/file.mp3", "rb");`
- Note: it isn't strictly platform independent...
 - Big Endian
 - Little Endian

Reading binary files

```
size_t fread(void *buf, size_t size, size_t count, FILE *f)
```

- buf is the place to store your data
 - size is the size of each element
 - count is the number of elements to be read
 - f is the file handle
- returns the number of elements read (should be count)

Writing to binary files

```
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *f)
```

- ptr – data source
 - size – size of each element
 - count – number of elements
 - f - file handle
- Returns number of elements written

Getting the size of a file...

```
#include <stdio.h>

void main (void)
{
    long lSize = 0;
    FILE *f;

    f = fopen ("adventures.txt", "r");
    if (f != NULL) {
        if (fseek(f, 0, SEEK_END) == 0) {
            lSize = ftell(f);
            printf ("Size of file: %ld\n", lSize);
            rewind(f);
        }
        fclose (f);
    }
}
```

There is no “get file properties” function, move the filepointer to the end of the file and check it's value

Getting the size of a file...

```
#include <sys/stat.h>
#include <stdio.h>

void main (void)
{
    struct stat sBuffer;
    int iRc;

    iRc = stat("adventures.txt", &sBuffer);
    if (iRc == 0) {
        printf ("Size of file: %ld\n", sBuffer.st_size);
    }
}
```

This function basically does the same as we did by moving the filepointer, for some such a “helper-function” might be more userfriendly?

See also <http://www.cplusplus.com/reference/cstdio/>

Other I/O

Named Pipe

- Essentially a FIFO
 - First In First Out - queue
- **unidirectional data channel**
- Simply a file on your machine
- Example :
 - `mkfifo myPipe`
 - See <https://linuxprograms.wordpress.com/tag/mkfifo/>
 - <https://www.softprayog.in/programming/interprocess-communication-using-fifos-in-linux>

Working with pipes

- Just as any files!
- Blocking I/O
 - Must be handled at both ends
- Typically used for using third-party programs and asynchronous communication between applications

Exam example – H21:03

Exam task (2021 exam)

- Download the file PG3401_Lecture_06_Hjemmeeksamen-14dager-H21_OPPG3.zip from Canvas, it contains source files

```

kasseapparat.c
kasseapparat.h
main.c
pgdbglog.c
pgdbglog.h

```

6.520	1.776	C Source	14/02/2022 12:...	8167FDCE
1.897	695	C/C++ Header	09/02/2022 14:...	B2604E0C
10.247	2.888	C Source	09/02/2022 14:...	0077173D
4.821	1.452	C Source	03/02/2022 09:...	2FE8F6E8
1.193	528	C/C++ Header	03/02/2022 09:...	898A2438

This zip file also includes a “debugger” code, we will cover that in lecture 7



Høyskolen
Kristiania